

Amortized Analysis

This is a technique used to analyze the average cost per operation over a sequence of operations even if some individual operations are expensive.

(i) Aggregate Method

Compute total cost of all operations - in divide by number of operations.

Formula:

$$\text{Cost} = \text{Total Cost} / \text{Number of operations}$$

Example:

let us say one algo has complexity $O(n \log n)$

$$\therefore \text{Amortized cost} = \frac{O(n \log n)}{n} = O(\log n)$$

For multiple Push and Pop time complexity:

$$O(n) + O(n) = 2O(n)$$

for one Push, Pop

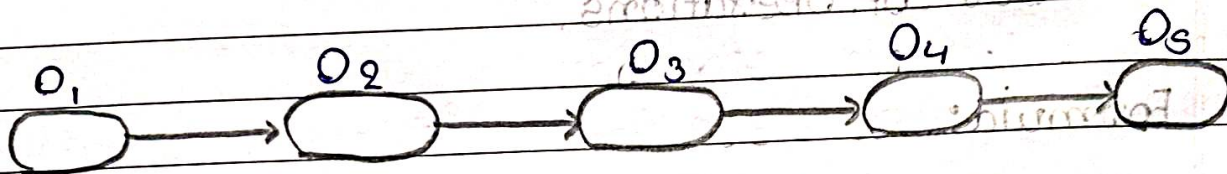
$$\text{cost} = \frac{O(n)}{n} = 1$$

cii) Accounting method

Assign extra cost to cheap operations. Save that cost to pay for expensive ones later.

Rules:

Amortized cost $\geq$ actual cost



Assumption (Sec)	2	3	2	4	5	16
Accountable (Sec)	3	2	2	4	5	17
	1	3	2	4	4	14

ciii) Potential Method

Measures "Stored Energy" in data structure.

cost = Actual cost + change in potential

$$\hat{c} = c + \phi_{\text{after}} - \phi_{\text{before}}$$

operation	Actual	Potential change	Amortized
Push	1	+1	2
Pop	1	-1	0

cost counting

```

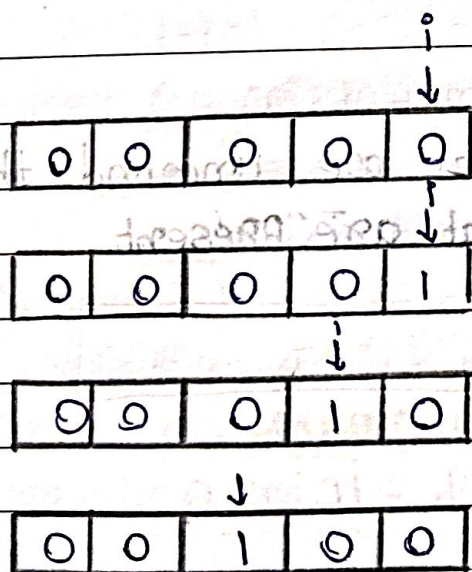
while i < len[A] & A[i] = 1
do A[i] ← 0
  i ← i + 1

```

```

if i < len[A]
then A[i] ← 1

```



we can make an Digit count based on this.

Time Complexity for recursion

```

★ fun(n) {
    if (n == 0) {
        return;
    }
    // processing
    int a = b + c
    // fun(n-1)
}
main() {
    fun(n);
}

```

fun(0)	$k_1 = 0$ bytes
⋮	
fun(n-1)	
fun(n)	$\rightarrow k$ bytes
main()	

now we unwind the stack that are present.

Stack

```

★ Printarray (int a[], int n)
    int a, b, c;
    if (n == 0) return;
    cout << *a << endl;  $\rightarrow k$ 
    Printarray(a+1, n-1)

```

$$f(n) = k + f(n-1)$$

①

$$T(n) = k + T(n-1)$$

$$T(n-1) = k + T(n-2)$$

$$\vdots$$

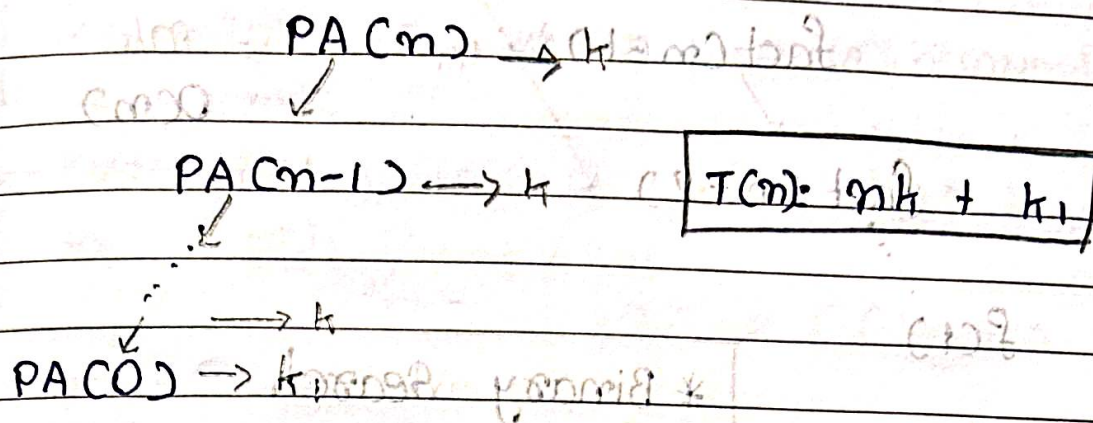
$$T(1) = k + T(0)$$

$$T(0) = k_1$$

$$T(n) = nk + k_1$$

$O(n)$

Recursive tree method



Space complexity

P.A.C(n)

$$SC(n) = O(n+1) + SC(n-1)$$

$= O(n)$

1	P.A.C(0)	m
2	⋮	⋮
3	⋮	⋮
⋮	⋮	⋮
n	P.A.C(n-1)	m
	P.A.C(n)	m
	mainC()	

* Factorial C(int n)

```

{
    if (n == 1)
        return 1;

```

return n * fact(n-1);

$$T(n) = k + T(n-1)$$

$$T(n-1) = k + T(n-2)$$

⋮

$$T(1) = k + T(0)$$

$$T(0) = k_1$$

$$nk + k_1$$

$$f(n) = n + f(n-1)$$

$$T(n) = k + T(n-1)$$

$$TO(Tn) = O(nk)$$

$$= O(n)$$

fact(n)

fact(n-1)

fact(n-2)

f(1)

* Binary Search

fact(1)

...

fact(n-1)

fact(n)

main()

n * m

$$F(n) = k + F(n/2)$$

$$T(n) = k + T(n/2)$$

$$T(n) = k + T(n/2)$$

$$T(n/2) = k + T(n/4)$$

$$T(n/4) = k + T(n/8)$$

$$\vdots$$

$$k(1) = k + T(1)$$

$$T(1) = k$$

$$T(n) = a * k$$

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \rightarrow \dots \rightarrow 1$$

$$2^a$$

$$\frac{n}{2^a} = 1 \quad a = \log(n)$$

Space

BS = 1	$n/2^0 = 1 \rightarrow k$
:	:
BS $n/4$	$n/4 \rightarrow k$
BS $n/2$	$n/2 \rightarrow k$
BS (n)	$n \rightarrow k$
main()	

It took space around

$$O(n \log n)$$

$$S.C = O(\log n)$$

Note:

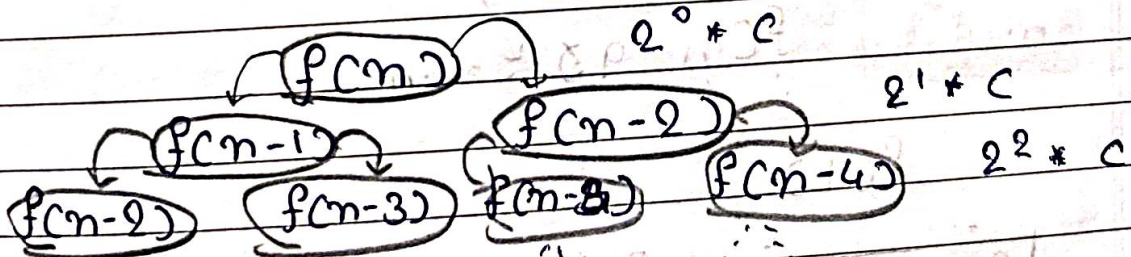
Hence it's a taking space it is reasonable to use iterative way instead of recursive way.

* Fibonacci Series

Behav Tarika

fib(int n)

if (n == 0 || n == 1) return;
return fib(n-1) + fib(n-2);



$$TC(n) \leq 2^0 + 2^1 + 2^2 + 2^3$$

$$TC(n) \leq C [2^0 + 2^1 + 2^2 + \dots + 2^{n-1}]$$

$$TC(n) \leq C [2^n - 1]$$

$$TC(n) \leq 2^n - 1$$

$$TC(n) = O(2^n)$$


```
fun C(int n) {
```

```
    if (n == 1) { return 1 }
```

```
    else {
```

```
        for (i = 1; i < n; i++) {
```

```
            for (j = 1; j < n; j++) {
```

```
                C(n-1)
```

```
                Print C"#" }
            }
```

```
        }
```

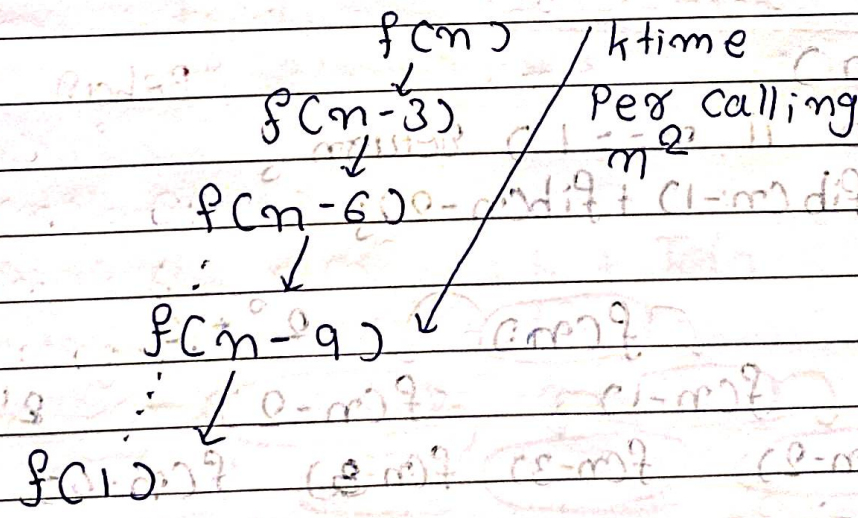
```
    }
```

at address fun(C(n-3)) print n-3

3 print address of fun(C(n-3))

$$T(n) = n^2 k + T(n-3)$$

#



$$T(n) = kn^2 + T(n-3)$$

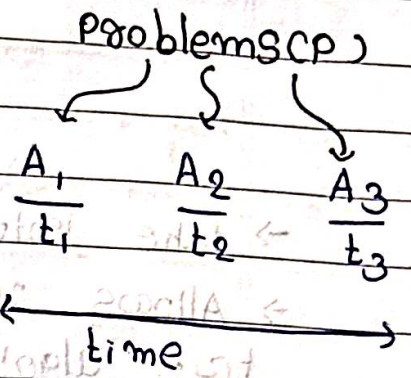
$$T(n) = kn^2 + k(n-3)^2 + k(n-6)^2 + \dots + k(1)^2$$

$$T(n) = kn^2$$

Need of analysis (why?)

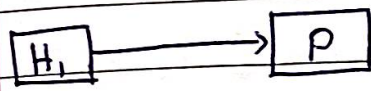
To Determine
Resource
Consumption
<Time + Consumption
Space>

Performance
Comparison
to find out
Efficient

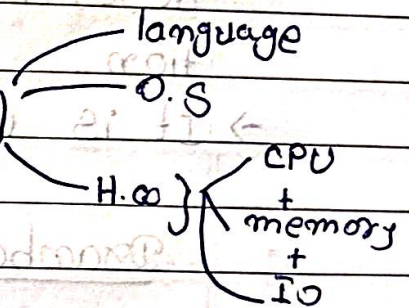


Methodology of Analysis (How?)

Time complexity



(1) $[x \leftarrow y + z]$



- walk : 30m
- cycle : 20m
- Bike : 10m
- Car : 6m

Platform Analysis

Aposterior Analysis

Aposterior Analysis

Advantage

Drawbacks/Limitation

- (1) It give exact Value in real units

- (1) It is difficulty to carry-out. (manual work)

- (2) cannot consider for cases of IP's (inputs)

- (3) non-uniformity

↳ not perform same of different time.

A Priori Analysis (Platform Independent)

Analytic Framework:

Tools

- Take into account all possible IP's
- Allows us to calculate the relative efficiency of two algo's in a way that is independent of Platform.
- Can be carried out by studying the high level description of algorithms without actual implementation
- It is easy to carry out.



Drawback:

- Will not give real/actual value in units
- Estimates / Appropriate / Approximate Value

Components of analytic framework:

- (1) A language (Pseudo) for describing Algorithms steps
- (2) A computation model that the Algorithms executes within it.
- (3) A metric for measuring algorithm running times.
- (4) An approach / Notation to characterize running time.

Step count method.

Algorithm test

```

{
  (1)  $x \leftarrow y + z$   $2 \text{ Unit}$ 
  (2) for  $i \leftarrow 1$  to  $n$ 
       $a \leftarrow b + c$ 
  (3) for  $i \leftarrow 1$  to  $n$ 
      for  $j \leftarrow 1$  to  $m$ 
           $c \leftarrow m * m$ 
}

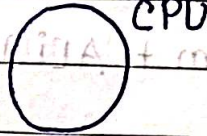
```

$4n^2 + 4n + 2 + 4n + 2 + 2$
 $T = 4n^2 + 8n + 6$

RAM Model of computation

memory

CHyper



Each function of n

Basic OP n

take one Unit time

Order of magnitude

It refers to freq/count of the fundamental in the Statement/Step;

(1) $x \leftarrow y + z$ 1 time

(2) for $i \leftarrow 1$ to n
 $a \leftarrow b + c$ n time

(3) for $i \leftarrow 1$ to n
 for $j \leftarrow 1$ to n
 $k \leftarrow k * z$ n^2 time

$$\text{Time} = n^2 + n + 1$$

Algo Sum(a, b, c)

{ int a, b, c

read (a, b); 1 time

if ($a < b$)

$c = a + b$ 2 time

else

$c = a * b$

print (c); 1 time

$$\text{Time} = 1 + 2 + 1$$

$$= 4$$


```
Sum(A, n) {
    int n, A[n]
    int i, Sum = 0
```

```
    for i ← 1 to n : n
        Sum = Sum + A[i]
```

```
    Print (Sum)
```

Time Complexity (as a function of Input Size n)

Polynomial

$(n^x) \ x > 0$

Ex: n, n^2, n^3

Exponential

a^n

Ex: $2^n, 4^n, n^n$

n	n^2	2^n
2	4	4
3	9	8
4	16	16
5	25	32
6	36	64
7	49	128

→ Polynomial function have lession rate of growth
(Take less time)

→ The objective is always to develop Algo for problems having Polynomial Time Complexity [Efficient]

Apriori Analysis

To determine the running time w.r.t. increasing input size n

To observe the behavior of the algorithm for a fixed input of size n

```
LS(A, n, x) {  
  int n, A[n]
```

1) Best case:

$$1+1+1 = C$$

$$O(1)$$

```
  int i;  
  for (i ← 1 to n)
```

2) worst case:

```
    if (x = A[i])
```

```
      print(i);
```

```
    exit;
```

```
  else
```

```
}
```

```
print("element not found")
```

```
}
```

worst case: The i/p class for which Algorithm takes max Time is w.c input & corresponding time is w.c Time.

Best case: The i/p class for which the algo takes min is B.C input & B.C time

Avg case: This derived in 3 steps

i) Enumerate all I/P classes.

ii) Determine the time for each IP-class.

$$B(n) \leq A(n) \leq \omega(n)$$

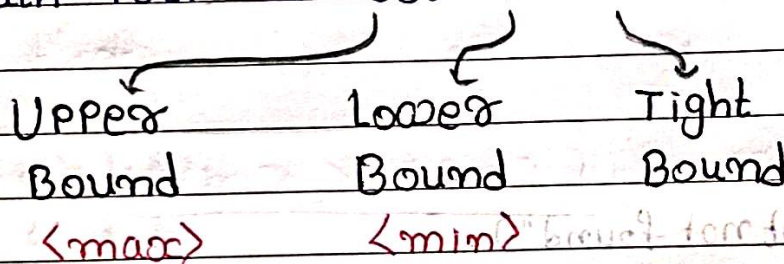
(1) $B(n) = A(n) = \omega(n)$ Merge Sort, Selection, He

(2) $B(n) < A(n) = \omega(n)$ Linear, Binary Search

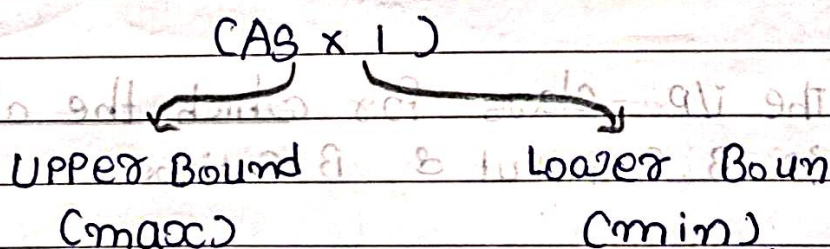
(3) $B(n) = A(n) < \omega(n)$ Quick Sort

Asymptotic Notations

Math Tool to obtain/rep^r bound of function



Representing Time & Space Complexity of algo by functions



ASN

Big $\rightarrow$ Big Oh: O $\rightarrow$ Little Oh: o

L. B $\rightarrow$ Big Omega: Ω $\rightarrow$ Little Omega: ω

$\rightarrow$ Big Theta: Θ

→ Let 'f' & 'g' be functions from the set of integer/real to real numbers:

(1) Big-Oh (O): Upper Bound

$f(n)$ is $O(g(n))$ if there exists some constants $c > 0$ and $n_0 > 0$

such that

$$f(n) \leq c g(n) \quad ; \quad n \geq n_0$$

example:

$$f(n) = 1 + n + n^2$$

$$1 + n \leq n^2$$

$$1 + n + n^2 \leq n^2 + n^2 + n^2$$

$$f(n) \leq 3n^2 \quad n \geq 1$$

$f(n)$ is $O(n^2)$.

$$Q \quad 1 + n + n^2 \leq a n^3 \quad ; \quad n \geq 1$$

$$1 + 1 + 1 \leq a \cdot 1 \cdot 1$$

$$1 + 2 + 4 \leq 5 \cdot 8$$

$$\therefore 7 \leq 40 \quad \checkmark$$

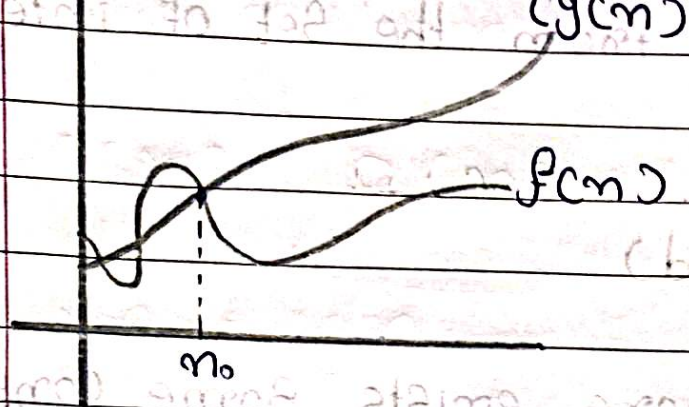
$f(n)$ is $O(n^2)$ → closest Upper Bound.

$f(n)$ is $O(n^3)$ → Loose Upper Bound.

$f(n)$ is $O(n^4)$

Note:

when ever determine the U.B & L.B, we should find that function 'g' which is closest to given functions.

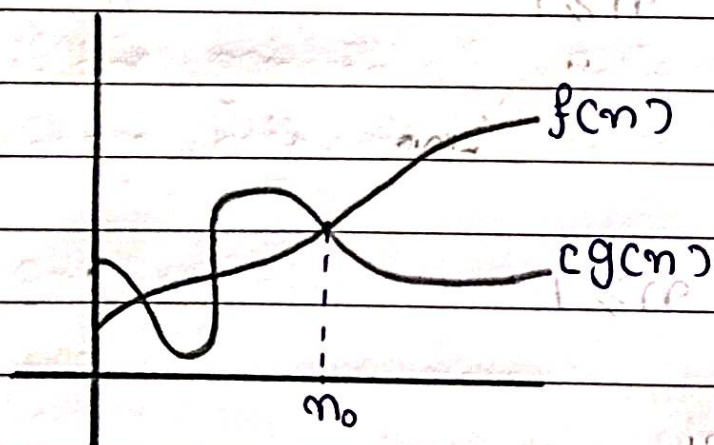


$$f(n) = O(g(n))$$

(2) Big - Omega : Lower Bound Ω

$f(n)$ is $\Omega(g(n))$ if exists const's 'c' & 'n₀' such that

$$f(n) \geq c g(n) \quad ; \quad n \geq n_0$$



example: $1+n+n^2 = \Omega(1), \Omega(n), \Omega(n^2)$

$$1+n+n^2 \geq 1 \quad ; \quad n \geq 1$$

$$1+n+n^2 \geq n \quad ; \quad n \geq 1$$

$$1+n+n^2 \geq 1 \cdot n^2 \quad ; \quad n \geq 1$$

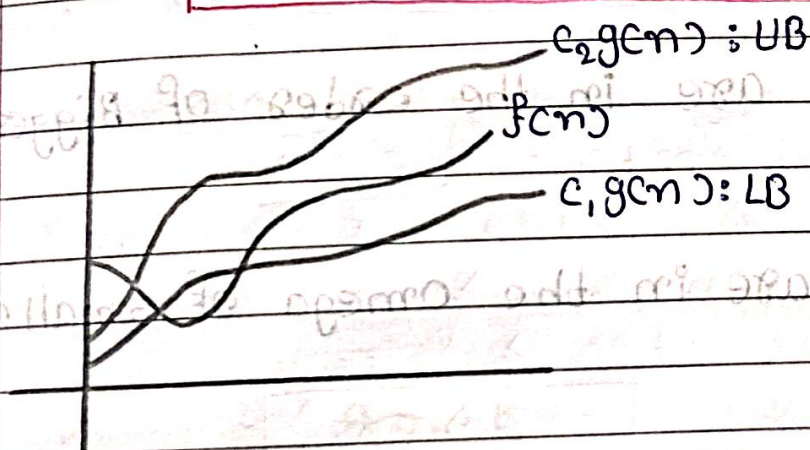
3) Theta (Θ): Tight Bound

$f(n)$ is $\Theta(g(n))$ if $f(n)$ is $O(g(n))$
 & $f(n)$ is $\Omega(g(n))$.

Example:

$$1+n+n^2 \begin{cases} O(n^2) \\ \Omega(n^2) \end{cases} \therefore \Theta(n^2)$$

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$



Q $f(n) = 1 + n + n^2 \begin{cases} O(n^2) \\ \Omega(n^2) \end{cases} \therefore \Theta(n^2)$

Q $f(n) = n \begin{cases} O(n) \\ \Omega(n) \end{cases} \therefore \Theta(n)$

Q $f(n) = 2^{100} \begin{cases} O(1) \\ \Omega(1) \end{cases} \therefore \Theta(1)$

Note: $c \begin{cases} O(1) \\ \Omega(1) \end{cases} \therefore \Theta(1)$
 Constant

(4) $f(n) = n + \log n$

$O(n) \quad \Omega(n) \therefore \Theta(n)$

(5) $\sqrt{n} + \log n$

$O(\sqrt{n}) \quad \Omega(\sqrt{n}) \therefore \Theta(\sqrt{n})$

(6) $f(n) = n^{1/\log n}$

O

Note:

→ Smaller functions are in the order of Bigger functions.

→ Bigger function are in the omega of smaller functions.

→ if the growth of both the functions are same than both each other theta.

Exponentials

log

$a^0 = 1$

$a^{-1} = 1/a$

$(a^m)^n = a^{mn}$

$a^m \cdot a^n = a^{m+n}$

$\log a^x = x \log a$

$\log(xy) = \log x + \log y$

$\log(\log(n))$

$a^{\log b} = b^{\log a}$

$\log^k n = (\log n)^k$

$$\log(x/y) : \log(x) - \log(y)$$

$$\therefore \log_b x = \frac{\log_a x}{\log_a b}$$

$$\therefore \log_b 1/a = -\log_b a$$

Arithmetic Series

$$\sum_{k=1}^n k = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

Geometric Series

$$\sum_{k=1}^n x^k = 1 + x + x^2 + \dots + x^n = \frac{x^{n+1} - 1}{x - 1}$$

Harmonic Series

$$\sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{n} \approx \log(n)$$

Geometric Sum formula for finite term is given

as:

if $r=1$; $S_n = n * a$

if $|r| < 1$; $S_n = \frac{a(1-r^n)}{1-r}$

if $|r| > 1$; $S_n = \frac{a(r^n-1)}{r-1}$

The geometric sum formula for infinite term is given as:

if $|x| < 1$ $S_{\infty} = \frac{a}{1-x}$

if $|x| > 1$; The series does not have sum

(1) $f(n) = 2^{\log_2 n}$ $Q. f(n) = n^{\frac{1}{\log_2 n}}$
 $O(n)$

(2) $f(n) = 2^{\sqrt{n} \log_2 n}$
 $= 2^{\log_2 n^{\sqrt{n}}}$
 $= n^{\sqrt{n}}$
 $O(n^{\sqrt{n}})$
 $Q. f(n) = n^{\frac{1}{\log_2 n}} = x$
 $\log (n^{\frac{1}{\log_2 n}}) = \log x$
 $\frac{1}{\log_2 n} \log_2 n = \log_2 x$
 $\log_2 x = 1$
 $x = 2^1$
 $x = 2$
 $x = O(1)$

(3) $f(n) = 2^{2n}$
 $O(2^{2n})$
 $O = (2^2)^n$
 $= 4^n > 2^n$
 $O(4^n)$

Q is $2^{n+1} = O(2^n)$?
 yes it is
 $2 \cdot 2^n \leq 4 \cdot 2^n$
 $f(n) \leq c g(n)$
 True.
 $\log_2 x = 1$
 $x = 2^1$
 $x = 2$

$$f(n) = \log n \quad g(n) = \sqrt{n}$$

$$g(n) \geq f(n)$$

~~g(n)~~ $f(n)$ is $O(g(n))$

$$f(n) = n^2 \quad ; \quad g(n) = n^3$$

$$n^2 \leq n^3$$

$$f(n) \leq c \cdot g(n)$$

$f(n)$ is $O(g(n))$ Hence, $f(n) = O(n^3)$

Increasing Order / Dominance Relation

★ Constant < logarithms < Poly < Exponential

★ $c < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 \dots 2^n < 4^n$
 $< n^n < n^{n^n} < n!$

$$Q \quad f(n) = \sum_{i=1}^n 1 = O(n)$$

$$Q \quad f(n) = \sum_{i=1}^n \frac{1}{2^i} = O(1 - \frac{1}{2^n})$$

$$Q \quad f(n) = \sum_{i=1}^n n \neq O(n) : n \sum_{i=1}^n 1 = O(n^2)$$

$$= O(1)$$

$$Q \quad f(n) = \sum_{i=1}^n i = O(n(n+1)/2) = O(n^2)$$

$$Q \quad f(n) = \sum_{i=1}^n i \cdot 2^i = (n-1)2^{n+1} + 2$$

$$= O(n \cdot 2^n)$$

$$Q \quad f(n) = \sum_{i=1}^n i^2 \neq O(n^2) = \frac{n \cdot (n+1) \cdot (2n+1)}{6} = O(n^3)$$

$$Q \quad f(n) = \sum_{i=1}^n i^3 \neq O(n^3) = \left[\frac{n \cdot (n+1)}{2} \right]^2 = O(n^4)$$

$$Q \quad f(n) = \sum_{i=1}^n 2^i = O(2^{n+1}) = (2^{n+1} - 2)$$

Small / Little Notations

- The Bound provided by Big-Notation, may or maynot be tight.
- The Bound provided by Small / Little notation is always not asymptotically tight. Loose Bound

(4) Small - Oh (o) Proper Upper Bound

$f(n)$ is $o(g(n))$ iff for all $c > 0$

$$f(n) < c \cdot g(n) \quad \text{, whenever } n > n_0$$

Example:

$$f(n) = n$$

$$o(n^2) \quad o(n) \quad o(n^2) \quad o(n^3) \quad o(n)$$

$$f(n) = 1 + n + n^2$$

$$o(n^3) \quad o(n^2) \quad o(n^2 \log n) \quad o(\Theta(n^2 \sqrt{n}))$$

(5) Small Omega (ω) : Proper Lower

$f(n)$ is $\omega(g(n))$ if $f(n)$ all c

$$f(n) > c \cdot g(n) \quad \text{; whenever } n > n_0$$

example

 $f(n)$ $\Omega(n)$ $\Omega(1)$ $\Omega(\log n)$ $\Omega(n^2)$ $\Theta(n)$ $\Theta(\log n)$ $\Theta(1)$ $\Theta(n^2)$ $\Theta(n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(n)$

$$(9) f(n) = \prod_{i=1}^n 1 = O(1)$$

$$(10) f(n) = \prod_{i=1}^n i = O(n!) = n \cdot n-1 \cdot n-2 \cdot \dots \cdot 1 < n^n$$

$$(11) f(n) = \sum_{i=1}^n \log(i) = O(n \log n)$$

$n! < n^n$
 $\Downarrow$
 $O(n^n)$ loose Bound

Stirling Approximation

$$n! \sim \sqrt{2\pi n} \cdot \left(\frac{n}{e}\right)^n \quad \text{vs} \quad n^n$$

$$\sqrt{2\pi} \cdot \sqrt{n} \cdot \left(\frac{n}{e}\right)^n \quad \text{vs} \quad n^n$$

$$c \cdot \sqrt{n} \quad \text{vs} \quad e^n$$

$$\sqrt{n} < e^n$$

$$n! \neq \Omega(n^n)$$

$$Q \quad n! = \Omega(n) ?$$

yes but not as tight bound as loose bound.

$$(11) f(n) = \sum_{i=1}^n \log i = \log(1) + \log(2) + \dots + \log(n)$$

$$= \log(1 + 2 + \dots + n)$$

$$= \log(n!)$$

$$n! \sim \sqrt{2\pi n} (n/e)^n$$

$$\begin{aligned} \log(n!) &= \log \sqrt{2\pi} + \frac{1}{2} \log(n) + n(\log n/e) \\ &= \log \sqrt{2\pi} + \frac{1}{2} \log(n) + n \log n + \log \frac{1}{e^n} \\ &= \left[c + \frac{1}{2} \log n + n \log n - n \log e \right] \end{aligned}$$

$$\Theta(n \log n) = \Omega(n \log n)$$

$$\Theta(\log(n!)) = \Theta(n \log n)$$

Another method

$$\begin{aligned} \log(n!) &= \sum_{i=1}^n \log i \approx \int_1^n \log x \, dx = [x(\log x - 1) + c]_1^n \\ &= [n \log n - n + c] \\ &\quad \underbrace{}_{\Theta(n \log n)} \quad \underbrace{}_{\Omega(n \log n)} \end{aligned}$$

$$\boxed{\Theta(n \log n)}$$

$$f(n) = \sum_{i=1}^n (i)^{1/2}$$

$$f(n) = \int_1^n x^{1/2} \, dx$$

$$\boxed{O(n\sqrt{n})}$$

$$= \frac{2}{3} [x^{3/2}]_1^n$$

$$= \frac{2}{3} (n^{3/2} - 1)$$

$$= \frac{2}{3} (n\sqrt{n} - 1)$$

PyQ $f(n) = \sum_{i=1}^n i^3 = x$; chice for x

~~CA~~ $\theta(n^4)$

~~CD~~ $O(n^5)$

~~CB~~ $\theta(n^5)$

~~CD~~ $\Omega(n^3)$

$$f(n) = \sum_{i=1}^n i^3$$

$$= \frac{(n(n+1))^2}{2}$$

$$= \frac{(n^2+n)^2}{2}$$

$$= O(n^4)$$

$$\theta(n^4)$$

$$\Omega(n^3)$$

$$f(n) \geq c \cdot g(n)$$

$$n^4 \geq c \cdot n^3$$

$$[n + n - n \cdot \text{pol}(n)] = \checkmark$$

$$[n \cdot \text{pol}(n)]$$

$$[n \cdot \text{pol}(n)]$$

Imp Remember

Properties of ASN:-

classmate

Date _____

Page _____

Analogy b/w real No's & ASN

Let a, b is real No's & f, g : +ve Functions.

(I) If $f(n)$ is $O(g(n))$

$$\Downarrow$$
$$a \leq b$$

(II) If $f(n)$ is $\Omega(g(n))$

$$\Downarrow$$
$$a \geq b$$

(III) If $f(n)$ is $\Theta(g(n))$

$$\Downarrow$$
$$a = b$$

(IV) If $f(n)$ is $O(g(n))$

$$\Downarrow$$
$$c = a < b$$

$$c \leq a < b$$

$$c > a < b$$

(V) If $f(n)$ is $\omega(g(n))$

$$\Downarrow$$
$$a > b$$

Discrete Properties of ASN

Properties \ ASN	O	Ω	Θ	o	ω
Reflexive	✓	✓	✓	X	X
Symmetric	X	X	✓	X	X
Transitive	✓	✓	✓	✓	✓
Transpose Symmetry	$\leftarrow$ if $f(n)$ is $O(g(n))$ then $g(n)$ is $\Omega(f(n))$		X	$\leftarrow$ if $f(n)$ is $o(g(n))$ then $g(n)$ is $\omega(f(n))$	$\rightarrow$ if $f(n)$ is $\omega(g(n))$ then $g(n)$ is $o(f(n))$

Trichotomy Property:

for any two real no's a, b

if ' f ' & ' g ' are +ve f'n's

(I) $f <_A g$ (II) $f \geq_A g$ (III) $f =_A g$

Does ASN / ASN functions obey Trichotomy property always?

→ No, Asymptotic Notations do not always obey the trichotomy property.

↳ This property fail when f^n are non-com
This usually happens with functions that oscill

Ex: $f(n) = n$ $g(n) = n^{1+\sin n}$
 $\sin(n) = 1$, $g(n) = n^2$ so $g(n) > f(n)$
 $\sin(n) = -1$, $g(n) = n^0$ so $f(n) > g(n)$

Arrange f^n in increasing order

2. $n^2 ; n \log n ; n\sqrt{n} ; e^n ; 2^n ; 1/n$

$$1/n < n \log n < n\sqrt{n} < n^2 < 2^n < e^n$$

2. $\frac{2^n}{e} ; \frac{n^{3/2}}{e} ; \frac{n \log n}{e} ; \frac{n^{\log n}}{e}$

$$n \log n < n^{3/2} < 2^n < n^{\log n}$$

~~$n \log n < n^{3/2} < n^{\log n}$~~
 ~~$n \log n < n^{\log n} < n^{3/2} < 2^n$~~

2. $\frac{n^{1/3}}{p} ; \frac{e^n}{e} ; \frac{n^{7/4}}{p} ; \frac{n \log^9 n}{p} ; \frac{(1.1001)^n}{e}$

$$n^{7/4} \quad n \log^9 n$$

$$(n \cdot n^{0.5} \cdot n^{0.25}) \quad n \log^9 n$$

$$\sqrt{n} \cdot n^{0.25} > (n \log n)^9$$

$$n^{1/3} < n \log^9 n < n^{7/4} < (1.1001)^n < e^n$$

10, $\sqrt{n}$, n , $\log_2 n$, $100/n$

$\hookrightarrow$
 10, $\frac{100}{n}$, $\log_2 n$, $\sqrt{n}$, n

$$n=3$$

$$3 \cdot 2^3$$

$$2^4 < 2^6$$

$$n=2$$

$$2 \cdot 2^2$$

$$2^3 < 2^4$$

Q $f(n) = n \cdot 2^n$; $g(n) = 4^n$

$$n \cdot 2^n < 2^{2n}$$

$f(n)$ is $O(g(n))$

Q 2^{2^n} ; $n!$; 4^n ; 2^n

$$n! < 2^n < 4^n < 2^{2^n} < n!$$

Q $2^{\log n}$, $(\log n)^2$, $\sqrt{\log n}$, $\log \log n$

$$\log \log n < \sqrt{\log n} < (\log n)^2 < 2^{\log n}$$

Q. let $\omega(n)$ and $A(n)$ respectively, the worst case & avg case running time of an algorithm with input size of n which is always TRUE?

- (a) $A(n) = O(\omega(n))$
- (b) $A(n) = \Omega(\omega(n))$
- (c) $A(n) = \Theta(\omega(n))$
- (d) $A(n) = O(\omega(n))$

Analysing Non-Recursive algorithms

Q. An element in an Array is called Leader if it is greater than all elements to the right of it. The Time complexity of the most efficient algorithm to print all leaders of the given Array of size 'n' is Linear time.

BF LEADER(A, n)

for $i \leftarrow 1$ to $n-1$

for $j \leftarrow i+1$ to n

{

if $(A[i] < A[j])$

break

}

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2} = O(n^2)$$

(i) Best case

Time : $(n-1) : O(n)$

(ii) worst case

Time : $O(n^2)$

Time Complexity Formulas for Recursive Algo

The time complexity of Recursive algorithm derived / represent as a recurrence equation;

→ Solve Recurrence to get the value of a Recurrence as a mathematical f^n (Poly, log, Expo, Const)

→ Apply : O, Ω, Θ

what $C(n)$

```
{
  if  $C(n=1)$  return;
```

```
  else
```

```
  {
```

```
    what  $C(n-1)$ ;
```

```
     $B(n)$ ;
```

```
  }
```

```
}
```

(i) Assume $B(n) = O(1)$

Let $T(n)$ repr. Time Complexity of what $C(n)$

$$T(n) = C; \quad n = 1$$

$$\rightarrow \boxed{= a + T(n-1) + b}$$

$$\boxed{T(n) = T(n-1) + C}$$

Solving recurrences

Back
Substitution
<Universal>
method

Repeated

Substitution

Master
Method
<Short>
Cut

Recursion
Tree

★ Back Substitution

$$T(n) = T(n-1) + d \quad (1)$$

$$T(n) = T(n-2) + d \quad (2)$$

$$T(n) = T(n-2) + d + d \quad (3)$$

$$= T(n-3) + d + 2d$$

$$= T(n-3) + 3d \quad (4)$$

$$\therefore d + d + d + \dots + d = (n-1)d$$

$$T(n) = T(n-k) + kd \quad (5)$$

$$\therefore n - k = 1$$

$$\therefore k = n - 1$$

$$T(n) = c + d(n-1)$$

$$= dn - d + c \quad (n \geq 1)$$

$$T(n) = \Theta(n)$$